

Reproducible Verdicts: Cross-Platform Determinism in Schedule Verification

Kurtis Brierley — Brierley Sovereign Group Ltd, Nottingham Technical White Paper · 26 September 2026 · Lattice117

Abstract

A verification tool's output is only evidence if a second party, on different hardware, can re-derive it. We describe Lattice117, a schedule feasibility and rest-rule verifier whose evaluation path uses Q16.16 fixed-point integers with no floating-point arithmetic, and which emits a SHA-256 digest over a canonical form of the integers it actually evaluated together with the verdict it reached. We report bit-identical digests across ten platform configurations spanning two independently written WebAssembly engine families — Google's V8 and Apple's JavaScriptCore — and native Rust on Linux, Windows and macOS.

The result we consider worth publishing is not that one. It is this: **making the computation deterministic was the easy half. Every reproducibility defect we found was in the canonicalisation — in deciding what to hash — and none of them was arithmetic.** We anatomise four such defects, each with a different mechanism, and a fifth of a different species in which the digest was stable everywhere and the verdict was wrong anyway. We propose a four-part taxonomy — representational, type-level, structural, semantic — argue that it is complete for digest-based verification, and show that each part requires a different control. A system can be perfectly deterministic and still produce irreproducible verdicts.

1. Introduction

A schedule audit is only useful if it can be re-run adversarially. If an employer produces a document saying a rota was checked and found compliant, the question a regulator, an insurer or a union representative will ask is not "what does your tool say?" but "what does your tool say *when I run it?*"

That is a stronger requirement than it first appears, and "the code is open" does not satisfy it. Reproducibility is a property of the whole pipeline — input handling, type coercion, serialisation, and the definition of the quantities being compared — not of the source alone. Two people can run identical source over identical data and disagree, and every way we found of making that happen had nothing to do with the arithmetic.

This paper is a report from building such a tool across five delivery surfaces: a native Rust CLI, an npm package, a browser page, a Microsoft Excel task pane and a Google Sheets sidebar, all over one WebAssembly engine. We set out to demonstrate cross-platform determinism. We found it, in the sense we expected and with less difficulty than anticipated. What consumed the effort — and what we think is the transferable result — was everything surrounding it.

2. Background: where non-determinism actually comes from

It is worth setting expectations honestly before the results, or the results look trivial.

Floating point is the usual suspect and is largely a red herring on modern 64-bit targets. IEEE-754 binary64 is fully specified, and both x86-64 and AArch64 implement it faithfully for the basic operations. The classic divergence sources are narrower than folklore suggests: x87 80-bit excess precision, which is a 32-bit x86 concern; and fused-multiply-add

contraction, where a compiler fuses `a*b+c` into one instruction with a single rounding instead of two. Neither arises in a parse-then-scale-then-compare-integers pipeline.

What does vary is less glamorous: library implementations of transcendental functions, reduction order under parallelism, hash iteration order, locale, line endings, and clocks.

The under-discussed one is serialisation. Two runs can agree on every number and disagree on the bytes that represent them. Nothing in the literature on reproducible builds or floating-point determinism prepares you for a digest that moves because a text file was checked out on a different operating system. That is §5.1.

3. System under test

Representation. Q16.16 fixed point on `i32`: one integer scaled by 65,536, giving a range of $-32,768 \dots +32,767$ and a resolution of $1/65,536$. Every quantity the engine compares — travel times, arrival times, window boundaries, rest durations, shortfalls — is one of these integers.

No floating point in the evaluation path. `crates/lattice117-verify/src/` and `crates/lattice117-wasm/src/` contain no `f32` or `f64`, enforced by a CI step that fails the build if either appears. Floats *are* permitted at the I/O boundary — `to_q16 / from_q16` in the CLI, `Math.round(Number(v) * 65536)` in JavaScript — and that boundary is precisely what §6's sweep exists to check. The distinction matters: the claim is not "no floats anywhere", it is "no floats where the verdict is decided."

Refusal over wrapping. Out-of-range input is rejected by name rather than silently wrapped. A wrapped value produces a *confident* verdict about a schedule that was never checked, which is worse than an error.

Five surfaces, one engine. The WebAssembly module is 21,525 bytes, has no `wasm-bindgen` layer, no dependencies, and opens no sockets. CI byte-compares the binary and the shared canonicalisation source across every surface.

4. Method

Fixtures. Two, both in the repository and both small enough to print (Appendix B): an 8-round fleet schedule (`demo/example-route-sheet.csv`) and a 4-person rota (`demo/example-rota.csv`). A third, `examples/infeasible.json`, exercises the native CLI.

The digest. SHA-256 over a canonical form of the Q16.16 integers actually evaluated, plus the verdict, plus — see §5.4 — the thresholds the verdict was measured against. It is emphatically **not** a signature; see §8.

Continuous integration. The digest is pinned. A matrix job runs the native binary on Ubuntu, Windows and macOS and fails the build if any platform disagrees. Further jobs assert that the WASM binary, the canonicalisation source, the rota rules and the licence check are byte-identical across surfaces; that the Sheets bundle is not stale; and that the rota digest is invariant across nine host timezones.

Device capture. `demo/determinism.html` runs both fixtures and a boundary sweep on whatever device opens it, then prints one evidence block with the device's reported environment and the digests it computed. Expected values are compiled in, so a mismatch reads as FAIL on the device rather than needing to be spotted by eye afterwards.

5. The central finding: five ways a deterministic system produced verdicts you could not rely on

§5.1–5.4 are **reproducibility failures** — the same input yielded different digests. §5.5 is a **different species**: the digest was stable everywhere and the verdict was wrong anyway. It was found not by a test failing but by applying §5.6's taxonomy forward, which is the best evidence we have that the taxonomy is worth something.

5.1 Representational — line endings changed the hash

The digest was taken over the input *text*. With no `.gitattributes`, Git rewrote LF to CRLF on the Windows checkout. Linux and macOS returned `611eef21...`; Windows returned `5467faa0...` — **with byte-identical Q16.16 integers in the violation**. Reproduced locally by feeding a CRLF copy to the Linux binary. Fixed by normalising CRLF to LF before hashing, which is a no-op on LF input, so previously published digests remained valid.

Lesson. Hashing a representation rather than a value imports every property of that representation, including the ones an operating system controls on your behalf.

5.2 Type-level — `600` and `'600'` disagreed

The npm package hashed values as supplied. A CSV or spreadsheet round-trip yields strings, so the same schedule, with the same verdict, produced two digests depending on how the caller happened to have parsed it. Fixed by hashing the Q16.16 integers the engine evaluated rather than the caller's inputs.

Lesson. Canonicalise *after* normalisation, not before. The boundary where types are coerced is the boundary where the canonical form should be taken.

5.3 Structural — three surfaces, three canonical forms

The browser page hashed raw inputs. The Excel task pane hashed a summary that **excluded the inputs entirely**, so two unrelated schedules that failed in the same shape produced the same digest. The npm package hashed Q16.16 integers. One schedule, three digests: `e249d90e...` against `8bad5881...` among others, measured before the fix.

This is the one that threatened the product rather than merely the paper. The commercial proposition — a certificate anyone can re-derive — is unsellable if re-deriving it on a different surface of the same product gives a different answer.

Fixed by extracting one shared `digest.js`, copied byte-identically to each surface and `cmp`-ed in CI.

Lesson. A digest whose canonicalisation is reimplemented per surface is not a digest. It is several digests wearing the same name.

5.4 Semantic — a verdict is meaningless without its thresholds

The rota check compares against configurable limits: 11 hours' inter-shift rest, a 20-minute break on shifts over 6 hours. A digest over shifts and outcomes alone could be reproduced under *different rules* and still match — the verdict "clear" is not a fact about a rota, it is a fact about a rota *and* a ruleset. The thresholds are therefore part of the canonical form.

This turns out to matter legally, not just theoretically. UK Working Time Regulations 1998 reg. 23(a) permits a collective or workforce agreement to modify or exclude the 11-hour entitlement outright, so a rota lawfully measured against a 9-hour rule is not an edge case — derogation is ordinary. A digest that did not pin its thresholds would let a derogated verdict and a statutory one collide. One that does pin them makes a derogated rota *auditable* rather than merely differently wrong.

Lesson. The canonical form must cover everything that could change the verdict, configuration included. Where the rules are lawfully variable, pinning them is not belt-and-braces — it is the only thing that makes the verdict mean anything.

5.5 The one the taxonomy predicted: daylight saving makes "eleven hours" ambiguous

This defect was found while writing the timezone caveat for §8 — that is, by applying the taxonomy of §5.6 rather than by a test failing.

The rota parser computes dates with an integer civil-date algorithm and never constructs a `Date`, because `new Date( '2026-09-28 06:00' )` is interpreted against the viewer's zone. That is what buys the timezone invariance reported in §6. It also means rest is measured in **civil** minutes, and on the night the clocks change, civil and elapsed time disagree:

Night (Europe/London)	Civil	Elapsed	Engine verdict
Sat 28 Mar 2026 22:00 → Sun 29 Mar 09:00	11h 00m	10h 00m	clear
Sat 24 Oct 2026 22:00 → Sun 25 Oct 09:00	11h 00m	12h 00m	clear

The autumn row is conservative and harmless. The spring row is a **false negative in the direction that matters**: the worker received ten hours' rest and the engine passed it.

The classification matters here.

- It is **not** a determinism defect. Every platform, and every host timezone including those with no such transition, returns the same verdict and the same digest for that night. Determinism is intact — and is exactly what makes the error *consistent* rather than intermittent.
- It is a **semantic** defect, §5.4's category, one level deeper. The canonical form covered everything that could change the verdict, but the *definition* of the quantity being compared against the threshold was never written down. "Eleven hours" was assumed unambiguous. Twice a year, it is not.

We then investigated which reading is correct rather than leaving it open. Three arguments converge on **elapsed time**. Reg. 10(1) is drafted in durations — "a rest period of not less than eleven consecutive hours in each 24-hour period" — and a civil reading must explain what a 24-hour period means on a spring-forward day that is 23 hours long. The Regulations are health-and-safety law whose protected interest is actual recovery, which a clock change does not supply. And the closest regulated regime settled it in practice: **drivers' hours under Regulation (EC) 561/2006 are recorded by digital tachographs that store UTC**, displaying local time but never recording it, precisely so that a clock change cannot alter a rest period.

That last point is a reproducibility argument as much as a legal one. The sector with the most enforcement and the most litigation over rest periods removed local time from the measurement entirely. **UTC is the canonical form for time.**

What we did about it:

- **Disclosed, not silently fixed.** The user-facing disclaimer now names the spring-forward night explicitly. It is not in the canonical form, so no published digest moved.
- **Not fixed with** `Date`. Computing elapsed time from the host zone would reintroduce the defect the integer parser exists to prevent.
- **The correct repair is a versioned schema change.** Elapsed rest needs a *declared* UTC offset per shift, supplied as data and refused by name when absent. By §5.4 that offset changes the verdict, so it must enter the canonical form — which moves every rota digest. That is a new canonical mode, not a patch. Specified, deliberately not implemented here.

Lesson, and the strongest in the paper. Eliminating a dependency does not eliminate the decision it was making. Refusing to consult the host timezone silently chose civil time over elapsed time. A removed dependency is a policy choice with nowhere left to be documented — which is why this survived five surfaces, four test suites and nine device captures.

5.6 Synthesis: a taxonomy

We propose four categories, and claim they are complete for digest-based verification:

Category	The question it answers	Failure when unanswered	Control
Representational	What bytes encode the value?	§5.1 — line endings moved the hash	Hash values, never their transport encoding
Type-level	What type is the value?	§5.2 — <code>600</code> $\neq$ <code>'600'</code>	Canonicalise after normalisation
Structural	What is included, and in what order?	§5.3 — three surfaces, three forms	One implementation, byte-compared in CI
Semantic	What does the value <i>mean</i> ?	§5.4, §5.5 — unpinned rules; undefined "hour"	Pin the rules; define the quantities

The completeness argument is that a digest is a function of *bytes*, *types*, *structure* and *meaning*, and there is nothing else for it to be a function of. Each category needs a different control, and the controls do not substitute for one another: §5.3's shared implementation would not have caught §5.1, and none of the first three would have caught §5.5. They are ordered by depth — each is invisible to the tests that catch the one before it.

6. Results

6.1 The determinism matrix

Platform	ISA	Runtime	Fleet	Rota	Sweep
Ubuntu (CI)	x86-64	native Rust	<code>611eef21...</code> *	—	—
Windows (CI)	x86-64	native Rust	<code>611eef21...</code> *	—	—
macOS (CI)	AArch64	native Rust	<code>611eef21...</code> *	—	—
Chromium / Linux	x86-64	V8 + WASM	<code>e249d90e...</code>	<code>83c66e6d...</code>	36,003 / 0
npm / Node 22	x86-64	V8 + WASM	<code>e249d90e...</code>	<code>83c66e6d...</code>	—
Excel task pane	x86-64	Edge WebView	<code>e249d90e...</code>	<code>83c66e6d...</code>	—
Google Sheets sidebar	x86-64	V8 + WASM	<code>e249d90e...</code>	<code>83c66e6d...</code>	—
Samsung Galaxy Tab A11	AArch64	Chrome 153 / V8	<code>e249d90e...</code>	<code>83c66e6d...</code>	36,003 / 0
Google Pixel 8 Pro	AArch64	Chrome 153 / V8	<code>e249d90e...</code>	<code>83c66e6d...</code>	36,003 / 0
iPhone SE (2020)	AArch64	Chrome 154 / JavaScriptCore	<code>e249d90e...</code>	<code>83c66e6d...</code>	36,003 / 0

* The native rows use `examples/infeasible.json`; the browser rows use `demo/example-route-sheet.csv`. These are **different inputs**, not a disagreement.

Nine device captures — three devices, three consecutive runs each, 26 September 2026. All nine reproduced both reference digests exactly and all nine reported zero divergences across 36,003 swept values. Blocks in Appendix A.

6.2 What the device captures are actually evidence of

"Three phones" is not the interesting number. Two facts in this dataset are load-bearing.

Two independently written WebAssembly implementations agree. The Android captures run V8 (Liftoff/TurboFan). The iPhone capture reports `CriOS/154` — Chrome's interface over WKWebView, because iOS in the United Kingdom permits no other engine — so its WebAssembly is executed by **JavaScriptCore** (BBQ/OMG) and its `Number / Math.round` by JSC. Two engine families, written by different vendors from the same specification, reaching byte-identical Q16.16 integers and byte-identical digests. This is the strongest single fact in the capture set and it was not something we set out to test.

Each device verified itself against ground truth, not merely against the others. The sweep compares the device's own `toQ()` against exact decimal arithmetic computed on that device with `BigInt`. Nine devices agreeing with each other could mean nine devices sharing a bug; nine devices agreeing with an exact oracle cannot.

What the devices do not add: a new instruction set. All three are AArch64, and the macOS CI runner already was — that runner is what surfaced six `E0133` errors in the NEON path under `#![forbid(unsafe_op_in_unsafe_fn)]`. Anyone counting "three new architectures" here would be counting wrong.

6.3 The boundary sweep

`Math.round(Number(v) * 65536)` is the only arithmetic outside WebAssembly, and the equivalent `to_q16` is the only float arithmetic in the CLI. The sweep checks that boundary against exact decimal arithmetic via `BigInt`, on-device:

Band	Count	Range
1 decimal place	6,001	0.0 ... 600.0
2 decimal places	6,001	0.00 ... 60.00
3 decimal places	20,001	0.000 ... 20.000
exact half-ULP offsets, <code>i + 0.5/65536</code> , to 10 dp	4,000	0 ... 3,999
Total	36,003	0 divergences, every device

The fourth band is where the teeth are: those inputs land exactly on `Math.round`'s tie-break, which is where a rounding-mode disagreement between engines would appear if one existed. It is 11% of the sweep and 100% of its value. The sweep is not 36,003 arbitrary values.

6.4 Timezone invariance

All nine device captures ran in `Europe/London`, so they do not test timezone independence at all. That gap is closed from the other side, in CI: the rota fixture digest is re-derived in nine host zones — including the 45-minute offset `Asia/Kathmandu`, the +13:45 `Pacific/Chatham`, and `Pacific/Kiritimati` across the date line — and is identical in all nine. The same suite asserts statically that the rota and digest modules construct no `Date` and read no locale or zone.

6.5 Elapsed time — an operability signal, not a benchmark

Device	Run 1	Run 2	Run 3
Galaxy Tab A11	525 ms	173 ms	158 ms
Pixel 8 Pro	180 ms	144 ms	132 ms
iPhone SE 2020	353 ms	446 ms	108 ms

Two fixtures plus a 36,003-value sweep complete in well under a second on every device, worst case 525 ms. That is what licenses the claim that verification is interactive on hardware a depot supervisor already owns.

These figures should not be over-read. First runs carry WebAssembly compilation and module fetch, and the iPhone's second run being slower than its first is scheduler and thermal noise. Three `performance.now()` samples are not a measurement. These figures bound the cost; they do not characterise it.

7. What is *not* a source of divergence

Two candidate sources of divergence were tested and found not to contribute.

The JavaScript float boundary. Across 36,003 realistic values, on four engine configurations, `Math.round(Number(v) * 65536)` produced bit-identical results to exact decimal arithmetic. Zero divergences.

Architecture, for a parse-then-scale pipeline. See §2.

We state plainly: **rewriting the parser to avoid the float boundary entirely would have added risk and changed nothing.** That is a negative result and it is worth publishing, because the instinct to eliminate every float is strong and, on this evidence, misdirected. The effort belongs in canonicalisation instead.

8. Limits, and what this does not prove

The following architectural and operational boundaries define what the verification digest does and does not establish:

Determinism is not correctness. A consistently wrong verdict is still wrong. §5.5 is precisely that case.

The digest is not a signature. It proves that the same input reached the same conclusion. It proves nothing about who produced it, when, or whether they were entitled to. Distinguishing a digest from an attestation matters; systems that blur the two are how "verified" comes to mean nothing.

Range. Q16.16 caps a schedule at roughly 22 days in minutes, or 9 hours in seconds. A wider type is not free: Q40.24 on `i64` doubles the memory of a dense distance matrix — for the 13,509-node TSPLIB instance `usa13509`, 0.73 GB becomes 1.46 GB. Above $2^{29} \approx 537$ million, staging a Q40.24 value through `f64` silently drops low fractional bits, since `f64` carries a 53-bit significand and Q40.24 keeps 24 fractional. Q16.16 never reaches that magnitude, which is exactly why §6.3's sweep found nothing. (Analysis only; parked as a specification, not implemented.)

Test set. Two fixtures, not a corpus.

No independent re-derivation. No external party has yet re-derived a digest. Until one has, the claim rests on our own CI.

Provenance of the captures. Device blocks are self-reported by the author, not attested. Nothing binds a pasted block to the hardware that produced it — the same limitation as the digest itself, one level up.

The user agent cannot identify the devices. Chrome's UA reduction freezes Android at `10` and the model at `K`, so the two Android blocks in Appendix A are textually identical but for the word `Mobile`. They are distinguished by GPU, core count and memory.

The clock-change false negative, §5.5, remains open by choice: disclosed, pinned by test, not fixed.

And the larger exposure runs the other way. WTR regs. 21, 22 and 24 disapply the daily rest entitlement for whole categories of worker. Reg. 22(1)(a) removes it outright when a shift worker changes shift — precisely the changeover a rota tool flags most often, though only for a "shift worker" as reg. 22(2) defines one and only "[s]ubject to regulation 24" — and reg. 21 covers security and surveillance work, continuity of service "as in hospitals", and foreseeable surges in agriculture,

tourism and postal services. So a *flagged* breach may be perfectly lawful, all year round, and the facts that decide it are not in the spreadsheet. No amount of determinism touches this. The tool is an indicative mathematical check, and this paper's determinism claims should never be read as compliance claims.

9. Related work

Reproducible Builds (reproducible-builds.org) establishes the general programme for bit-identical artefacts from identical sources, and is the closest methodological ancestor; our contribution is to apply the same standard to a *verdict* rather than a binary. Goldberg's *What Every Computer Scientist Should Know About Floating-Point Arithmetic* (1991) remains the reference for §2's distinction between specified and unspecified float behaviour. Certificate Transparency (RFC 6962) and Sigstore supply the vocabulary for §8's distinction between a digest and an attestation. For the problem domain, the SINTEF Solomon VRPTW benchmark set and the TSPLIB instances referenced in §8 are the standard corpora.

10. Reproduction

The results in this paper can be reproduced from source using the following commands:

```
git clone https://github.com/KABCTTRHC/lattice117-verify
cd lattice117-verify

cargo test --all
./target/release/lattice117-audit --input examples/infeasible.json --json

# Expected verdict digest:
# 611eef21de58485fe4727b74f54f4f6a06e1e1bf9567384c989a1e75b5cac085

# Rest and break boundaries, nine-timezone invariance, free-tier caps,
# and Sheets bundle parity:
node tests/rota.test.mjs
node tests/timezone.test.mjs
node tests/freetier.test.mjs
node tests/sheets.test.mjs

# Install from npm:
npm install lattice117-verify
```

On any device, open <https://kabcttrhc.github.io/lattice117-verify/determinism.html> and compare the block it prints against Appendix A.

Reference values:

Fixture	Digest
demo/example-route-sheet.csv	e249d90ef7b895243e48c9f8315e3fe3a3ab02732604a898c2c6922fda24888d
demo/example-rota.csv	83c66e6dfdd2f1b304364f65a102397b85bf1c225396c2dbd36df9272c92f3cf
examples/infeasible.json	611eef21de58485fe4727b74f54f4f6a06e1e1bf9567384c989a1e75b5cac085

Appendix A — evidence blocks

Verbatim as emitted by `demo/determinism.html` . Nothing retyped or summarised. Captured 26 September 2026.

A.1 — Samsung Galaxy Tab A11 (Chrome 153, V8)

LATTICE117 DETERMINISM EVIDENCE

captured 2026-09-26T10:55:56.755Z

DEVICE

user agent Mozilla/5.0 (Linux; Android 10; K) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/153.0.0.0 Safari/537.36

platform Linux armv81

cores 8 memory 4 GB

screen 1007x601 @ dpr 1.3312500715255737

gpu ANGLE (ARM, Mali-G57 MC2, OpenGL ES 3.2)

wasm Memory=yes Global=yes Table=yes instantiateStreaming=yes

timezone Europe/London

FLEET FIXTURE example-route-sheet.csv

rounds 8 checked, 5 achievable

engine 21525 bytes

digest e249d90ef7b895243e48c9f8315e3fe3a3ab02732604a898c2c6922fda24888d

expected e249d90ef7b895243e48c9f8315e3fe3a3ab02732604a898c2c6922fda24888d

result MATCH

ROTA FIXTURE example-rota.csv

staff 4 checked, 2 clear, 2 flagged

digest 83c66e6dfdd2f1b304364f65a102397b85bf1c225396c2dbd36df9272c92f3cf

expected 83c66e6dfdd2f1b304364f65a102397b85bf1c225396c2dbd36df9272c92f3cf

result MATCH

Q16.16 BOUNDARY SWEEP toQ() against exact BigInt decimal

values 36003

divergences 0

result MATCH

ELAPSED 525 ms

OVERALL DETERMINISTIC

LATTICE117 DETERMINISM EVIDENCE

captured 2026-09-26T10:57:13.368Z

DEVICE

user agent Mozilla/5.0 (Linux; Android 10; K) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/153.0.0.0 Safari/537.36

platform Linux armv81

cores 8 memory 4 GB

screen 1007x601 @ dpr 1.3312500715255737

gpu ANGLE (ARM, Mali-G57 MC2, OpenGL ES 3.2)

wasm Memory=yes Global=yes Table=yes instantiateStreaming=yes

timezone Europe/London

FLEET FIXTURE example-route-sheet.csv

rounds 8 checked, 5 achievable

engine 21525 bytes

digest e249d90ef7b895243e48c9f8315e3fe3a3ab02732604a898c2c6922fda24888d

expected e249d90ef7b895243e48c9f8315e3fe3a3ab02732604a898c2c6922fda24888d

result MATCH

ROTA FIXTURE example-rota.csv

staff 4 checked, 2 clear, 2 flagged

digest 83c66e6dfdd2f1b304364f65a102397b85bf1c225396c2dbd36df9272c92f3cf

expected 83c66e6dfdd2f1b304364f65a102397b85bf1c225396c2dbd36df9272c92f3cf

result MATCH

Q16.16 BOUNDARY SWEEP toQ() against exact BigInt decimal

values 36003

divergences 0

result MATCH

ELAPSED 173 ms

OVERALL DETERMINISTIC

LATTICE117 DETERMINISM EVIDENCE

captured 2026-09-26T10:57:42.090Z

DEVICE

user agent Mozilla/5.0 (Linux; Android 10; K) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/153.0.0.0 Safari/537.36

platform Linux armv81

cores 8 memory 4 GB

screen 1007x601 @ dpr 1.3312500715255737

gpu ANGLE (ARM, Mali-G57 MC2, OpenGL ES 3.2)

wasm Memory=yes Global=yes Table=yes instantiateStreaming=yes

timezone Europe/London

FLEET FIXTURE example-route-sheet.csv

rounds 8 checked, 5 achievable

engine 21525 bytes

digest e249d90ef7b895243e48c9f8315e3fe3a3ab02732604a898c2c6922fda24888d

expected e249d90ef7b895243e48c9f8315e3fe3a3ab02732604a898c2c6922fda24888d

result MATCH

ROTA FIXTURE example-rota.csv

staff 4 checked, 2 clear, 2 flagged

digest 83c66e6dfdd2f1b304364f65a102397b85bf1c225396c2dbd36df9272c92f3cf

expected 83c66e6dfdd2f1b304364f65a102397b85bf1c225396c2dbd36df9272c92f3cf

result MATCH

Q16.16 BOUNDARY SWEEP toQ() against exact BigInt decimal

values 36003

divergences 0

result MATCH

ELAPSED 158 ms

OVERALL DETERMINISTIC

A.2 — Google Pixel 8 Pro (Chrome 153, V8)

LATTICE117 DETERMINISM EVIDENCE

captured 2026-09-26T11:00:44.273Z

DEVICE

user agent Mozilla/5.0 (Linux; Android 10; K) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/153.0.0.0
Mobile Safari/537.36

platform Linux armv81
cores 9 memory 8 GB
screen 448x998 @ dpr 2.25
gpu ANGLE (ARM, Mali-G715, OpenGL ES 3.2)
wasm Memory=yes Global=yes Table=yes instantiateStreaming=yes
timezone Europe/London

FLEET FIXTURE example-route-sheet.csv

rounds 8 checked, 5 achievable
engine 21525 bytes
digest e249d90ef7b895243e48c9f8315e3fe3a3ab02732604a898c2c6922fda24888d
expected e249d90ef7b895243e48c9f8315e3fe3a3ab02732604a898c2c6922fda24888d
result MATCH

ROTA FIXTURE example-rota.csv

staff 4 checked, 2 clear, 2 flagged
digest 83c66e6dfdd2f1b304364f65a102397b85bf1c225396c2dbd36df9272c92f3cf
expected 83c66e6dfdd2f1b304364f65a102397b85bf1c225396c2dbd36df9272c92f3cf
result MATCH

Q16.16 BOUNDARY SWEEP toQ() against exact BigInt decimal

values 36003
divergences 0
result MATCH

ELAPSED 180 ms

OVERALL DETERMINISTIC

LATTICE117 DETERMINISM EVIDENCE

captured 2026-09-26T11:01:08.110Z

DEVICE

user agent Mozilla/5.0 (Linux; Android 10; K) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/153.0.0.0
Mobile Safari/537.36

platform Linux armv81

cores 9 memory 8 GB

screen 448x998 @ dpr 2.25

gpu ANGLE (ARM, Mali-G715, OpenGL ES 3.2)

wasm Memory=yes Global=yes Table=yes instantiateStreaming=yes

timezone Europe/London

FLEET FIXTURE example-route-sheet.csv

rounds 8 checked, 5 achievable

engine 21525 bytes

digest e249d90ef7b895243e48c9f8315e3fe3a3ab02732604a898c2c6922fda24888d

expected e249d90ef7b895243e48c9f8315e3fe3a3ab02732604a898c2c6922fda24888d

result MATCH

ROTA FIXTURE example-rota.csv

staff 4 checked, 2 clear, 2 flagged

digest 83c66e6dfdd2f1b304364f65a102397b85bf1c225396c2dbd36df9272c92f3cf

expected 83c66e6dfdd2f1b304364f65a102397b85bf1c225396c2dbd36df9272c92f3cf

result MATCH

Q16.16 BOUNDARY SWEEP toQ() against exact BigInt decimal

values 36003

divergences 0

result MATCH

ELAPSED 144 ms

OVERALL DETERMINISTIC

LATTICE117 DETERMINISM EVIDENCE

captured 2026-09-26T11:01:28.196Z

DEVICE

user agent Mozilla/5.0 (Linux; Android 10; K) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/153.0.0.0
Mobile Safari/537.36

platform Linux armv81

cores 9 memory 8 GB

screen 448x998 @ dpr 2.25

gpu ANGLE (ARM, Mali-G715, OpenGL ES 3.2)

wasm Memory=yes Global=yes Table=yes instantiateStreaming=yes

timezone Europe/London

FLEET FIXTURE example-route-sheet.csv

rounds 8 checked, 5 achievable

engine 21525 bytes

digest e249d90ef7b895243e48c9f8315e3fe3a3ab02732604a898c2c6922fda24888d

expected e249d90ef7b895243e48c9f8315e3fe3a3ab02732604a898c2c6922fda24888d

result MATCH

ROTA FIXTURE example-rota.csv

staff 4 checked, 2 clear, 2 flagged

digest 83c66e6dfdd2f1b304364f65a102397b85bf1c225396c2dbd36df9272c92f3cf

expected 83c66e6dfdd2f1b304364f65a102397b85bf1c225396c2dbd36df9272c92f3cf

result MATCH

Q16.16 BOUNDARY SWEEP toQ() against exact BigInt decimal

values 36003

divergences 0

result MATCH

ELAPSED 132 ms

OVERALL DETERMINISTIC

A.3 — iPhone SE 2020, iOS 26.6.2 (Chrome 154 / CriOS — JavaScriptCore)

LATTICE117 DETERMINISM EVIDENCE

captured 2026-09-26T11:03:02.379Z

DEVICE

user agent Mozilla/5.0 (iPhone; CPU iPhone OS 26_6_2 like Mac OS X) AppleWebKit/605.1.15 (KHTML, like Gecko) CriOS/154.0.8037.55 Mobile/15E148 Safari/604.1

platform iPhone

cores 4 memory ? GB

screen 375x667 @ dpr 2

gpu Apple GPU

wasm Memory=yes Global=yes Table=yes instantiateStreaming=yes

timezone Europe/London

FLEET FIXTURE example-route-sheet.csv

rounds 8 checked, 5 achievable

engine 21525 bytes

digest e249d90ef7b895243e48c9f8315e3fe3a3ab02732604a898c2c6922fda24888d

expected e249d90ef7b895243e48c9f8315e3fe3a3ab02732604a898c2c6922fda24888d

result MATCH

ROTA FIXTURE example-rota.csv

staff 4 checked, 2 clear, 2 flagged

digest 83c66e6dfdd2f1b304364f65a102397b85bf1c225396c2dbd36df9272c92f3cf

expected 83c66e6dfdd2f1b304364f65a102397b85bf1c225396c2dbd36df9272c92f3cf

result MATCH

Q16.16 BOUNDARY SWEEP toQ() against exact BigInt decimal

values 36003

divergences 0

result MATCH

ELAPSED 353 ms

OVERALL DETERMINISTIC

LATTICE117 DETERMINISM EVIDENCE

captured 2026-09-26T11:03:16.458Z

DEVICE

user agent Mozilla/5.0 (iPhone; CPU iPhone OS 26_6_2 like Mac OS X) AppleWebKit/605.1.15 (KHTML, like Gecko) CriOS/154.0.8037.55 Mobile/15E148 Safari/604.1

platform iPhone
cores 4 memory ? GB
screen 375x667 @ dpr 2
gpu Apple GPU
wasm Memory=yes Global=yes Table=yes instantiateStreaming=yes
timezone Europe/London

FLEET FIXTURE example-route-sheet.csv

rounds 8 checked, 5 achievable
engine 21525 bytes
digest e249d90ef7b895243e48c9f8315e3fe3a3ab02732604a898c2c6922fda24888d
expected e249d90ef7b895243e48c9f8315e3fe3a3ab02732604a898c2c6922fda24888d
result MATCH

ROTA FIXTURE example-rota.csv

staff 4 checked, 2 clear, 2 flagged
digest 83c66e6dfdd2f1b304364f65a102397b85bf1c225396c2dbd36df9272c92f3cf
expected 83c66e6dfdd2f1b304364f65a102397b85bf1c225396c2dbd36df9272c92f3cf
result MATCH

Q16.16 BOUNDARY SWEEP toQ() against exact BigInt decimal

values 36003
divergences 0
result MATCH

ELAPSED 446 ms

OVERALL DETERMINISTIC

LATTICE117 DETERMINISM EVIDENCE

captured 2026-09-26T11:03:26.270Z

DEVICE

user agent Mozilla/5.0 (iPhone; CPU iPhone OS 26_6_2 like Mac OS X) AppleWebKit/605.1.15 (KHTML, like Gecko) CriOS/154.0.8037.55 Mobile/15E148 Safari/604.1

platform iPhone
cores 4 memory ? GB
screen 375x667 @ dpr 2
gpu Apple GPU
wasm Memory=yes Global=yes Table=yes instantiateStreaming=yes
timezone Europe/London

FLEET FIXTURE example-route-sheet.csv

rounds 8 checked, 5 achievable
engine 21525 bytes
digest e249d90ef7b895243e48c9f8315e3fe3a3ab02732604a898c2c6922fda24888d
expected e249d90ef7b895243e48c9f8315e3fe3a3ab02732604a898c2c6922fda24888d
result MATCH

ROTA FIXTURE example-rota.csv

staff 4 checked, 2 clear, 2 flagged
digest 83c66e6dfdd2f1b304364f65a102397b85bf1c225396c2dbd36df9272c92f3cf
expected 83c66e6dfdd2f1b304364f65a102397b85bf1c225396c2dbd36df9272c92f3cf
result MATCH

Q16.16 BOUNDARY SWEEP toQ() against exact BigInt decimal

values 36003
divergences 0
result MATCH

ELAPSED 108 ms

OVERALL DETERMINISTIC

Appendix B — the fixtures

demo/example-rota.csv

```
Staff Name,Date,Shift Start,Shift End,Break (mins)
Nadia Okafor,2026-09-28,14:00,22:00,30
Nadia Okafor,2026-09-29,06:00,14:00,30
Tom Whitmore,2026-09-28,07:00,15:30,15
Tom Whitmore,2026-09-29,09:00,17:00,30
Priya Raman,2026-09-28,08:00,16:00,30
Priya Raman,2026-09-29,08:00,16:00,30
Marcus Bell,2026-09-28,22:00,06:00,30
Marcus Bell,2026-09-29,20:00,04:00,30
```

```

vehicle,seq,stop>window_open>window_close,travel_mins_from_previous
VAN-01,0,DEPOT,0,600,0
VAN-01,1,Bilborough,0,57,22.3
VAN-01,2,Beeston,0,68,10.9
VAN-01,3,Clifton,0,74,5.8
VAN-01,4,DEPOT,0,600,21.0
VAN-02,0,DEPOT,0,600,0
VAN-02,1,W-Bridgford,0,42,7.1
VAN-02,2,Arnold,0,18,17.1
VAN-02,3,Hucknall,0,78,19.4
VAN-02,4,DEPOT,0,600,32.8
VAN-03,0,DEPOT,0,600,0
VAN-03,1,Carlton,0,53,18.8
VAN-03,2,Gedling,0,69,15.4
VAN-03,3,Stapleford,0,93,24.4
VAN-03,4,DEPOT,0,600,27.0
VAN-04,0,DEPOT,0,600,0
VAN-04,1,Long-Eaton,0,46,11.2
VAN-04,2,Ilkeston,0,63,17.5
VAN-04,3,Kimberley,0,84,20.7
VAN-04,4,DEPOT,0,600,6.3
VAN-05,0,DEPOT,0,600,0
VAN-05,1,Bulwell,0,64,29.3
VAN-05,2,Sherwood,0,74,51.1
VAN-05,3,Mapperley,0,196,42.5
VAN-05,4,DEPOT,0,600,22.2
VAN-06,0,DEPOT,0,600,0
VAN-06,1,Basford,0,63,28.2
VAN-06,2,Wollaton,0,83,20.0
VAN-06,3,Lenton,0,92,9.3
VAN-06,4,DEPOT,0,600,17.2
VAN-07,0,DEPOT,0,600,0
VAN-07,1,Sneinton,0,57,22.1
VAN-07,2,Colwick,0,26,10.4
VAN-07,3,Netherfield,0,124,45.2
VAN-07,4,DEPOT,0,600,27.2
VAN-08,0,DEPOT,0,600,0
VAN-08,1,Ruddington,0,56,21.4
VAN-08,2,Keyworth,0,88,32.3
VAN-08,3,Cotgrave,0,154,43.0
VAN-08,4,DEPOT,0,600,30.2

```